МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО–КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

Отчет по лабораторной работе № 1
«Введение в нейронные сети»
по дисциплине «Основы нейронных сетей»

Выполнил:
Студент группы ИВТ–б–о–23–1:
Даниелян Д.Э. « » _____20__г.
Подпись_____
Работа защищена « »_____20__г.
Проверил:
доцент департамента цифровых, робототехнических систем и электроники института перспективной инженерии
Воронкин Р.А._____(подпись)

Ставрополь, 2026.

Тема: Введение в нейронные сети.

Цель работы: освоение навыков по основам создания нейронных сетей и их более глубокого изучения.

Конспект теоретического материала:

Технология искусственного интеллекта

Профессия разработчика ИИ – №1 в Китае и США. В России входит в топ-5, но плавно идет к первому месту.

Различают два вида искусственного интеллекта: сильный и слабый.

- **Сильный ИИ** – разумное существо, которое, как вы видели в фильмах и читали в книгах, живет, осознает себя, чувствует, думает. В общем, имеет свое сознание и цели. И, если верить произведениям искусства, хочет уничтожить человечество. Но не бойтесь, он еще не скоро появится. Минимум лет через 30.
- **Слабый ИИ** – не смущайтесь от этого слова, это прикладная программа, которая решает конкретную задачу. Такие как автопилот автомобиля Tesla, распознавание голоса при использовании «ОК Google» или «Привет, Siri», оценка стоимости квартиры, подсчет людей на стройке. Это конкретные нейронки, которые решают конкретные бизнес-задачи. Именно с ними мы и будем работать.

Сферы применения

Почему ИИ относят в отдельную категорию, ведь есть программирование? Есть различные программы, такие как Zoom, Word, браузеры.

Различают интеллектуальные и неинтеллектуальные задачи.

Неинтеллектуальные задачи – это последовательность действий, выполняемых по определенному алгоритму. Например, беседа по видеосвязи. При ней видеокамера захватывает изображение и передает ее в ПК, где оно

сжимается, чтобы занимать меньше места, нарезается на кусочки и передается на роутер. Дальше через разные страны эти пакеты информации передаются в ПК, объединяются, декодируются, и собеседники видят картинку. Это очень сложная технология. Человечество тысячелетиями шло от добычи руды, разведения огня и до создания протокола IPv6 для интернета, чтобы освоить ее. Но это строго детерминированный алгоритм.

Сейчас основная область применения ИИ – это **интеллектуальные задачи**.

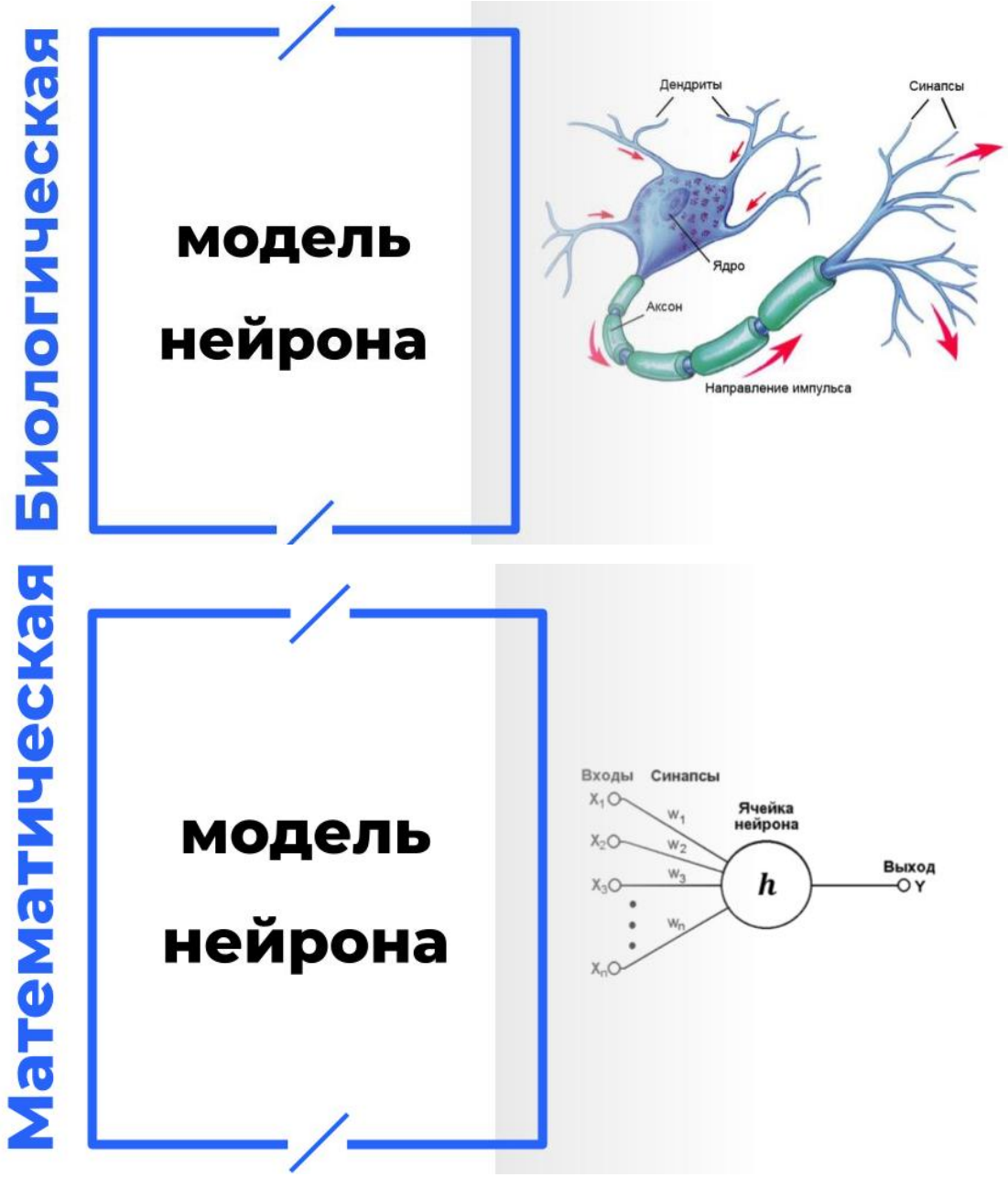
Жизненный цикл технологии

У каждой технологии есть свой цикл жизни. По оценке экспертов, ИИ будет расти до 2050 года, то есть еще 29 лет. А к 2070 искусственный интеллект станет для человека частью жизни, как компьютеры и мобильные телефоны.

Нейронные сети. Биологическая и математическая модели нейрона

Есть ядро нейрона и дендриты, по которым приходят сигналы от других нейронов. После того, как количество сигналов достигает определенной отметки, нейрон выстреливает импульс дальше по своим синапсам. Синапсы соединены с дендритами других нейронов.

В зависимости от активности нейрона может передаваться как весь импульс, так и его часть. Бывает, что в ответ на сигналы нейрон тормозится и импульс совсем не проходит.



Ячейка нейрона - это тело нейрона, где накапливается сигнал, **входы X** – насколько активизировались нейроны, с которыми он связан (в нейробиологии называют "возбуждающий/тормозящий постсинаптический потенциал"), а **веса (синапсы)** – это насколько сильны связи.

Выход ячейки нейрона (h) равен сумме произведений входов

(X1,…,Xn) и весов (w1,…,wn). Формула выглядит так:

$$h = X_1 * w_1 + X_2 * w_2 + X_3 * w_3 + \ldots + X_n * w_n$$

Вход X1 умножаем на вес w1, вход X2 умножаем на вес w2, и так каждый вход умножаем на соответствующий ему вес. Полученные значения складываем и получаем результирующее значение h.

Факт: вес – это сила связи между нейронами. Он определяет влияние входа на выход.

При обучении нейронные сети самостоятельно подбирают веса.

График ошибки в таком случае выглядит как пустыня с барханами. И веса, как шарики, движутся под действием силы тяжести, стремясь к минимуму ошибки.

В этом нейронке помогает специальный **алгоритм обратного распространения ошибки**. Это измененный **метод градиентного спуска** - математические формулы, которые вычисляют, как изменить веса, чтобы быстрее прийти к минимальной ошибке (а тем самым - к наилучшей точности).

Полносвязный (линейный, Dense) слой нейронов

Один из видов слоев: полносвязный (или линейный) слой, по-английский - Dense ("плотный") слой. Он является самым базовым и часто встречается почти в любой модели нейронной сети.

Выбор ускорителя

Google Colaboratory - ресурс компании Google, который предоставляет бесплатные видеокарты для обучения нейронных сетей

Но важно в каждом ноутбуке **включить** эту опцию (задействовать видеокарту).

Нейрон смещения или **bias-нейрон** — это третий вид нейронов, используемый в большинстве нейросетей. Это гиперпараметр, его наличие/отсутствие определяется вами.

Особенность этого типа нейронов заключается в том, что его вход и выход всегда равняются 1 и они никогда не имеют входных синапсов. Нейроны смещения могут, либо присутствовать в нейронной сети по одному на слое, либо полностью отсутствовать, 50/50 быть не может.

Соединения у нейронов смещения такие же, как у обычных нейронов — со всеми нейронами следующего уровня, за исключением того, что синапсов между двумя bias нейронами быть не может. Следовательно, их можно размещать на входном слое и всех скрытых слоях, но никак не на выходном слое, так как им попросту не с чем будет формировать связь.

Применение:

- дает возможность изменять выходной результат, путем сдвига графика функции активации вправо или влево.
- помогают в том случае, когда все входные нейроны получают на вход 0 и независимо от того какие у них веса, они все передадут на следующий слой 0, но не в случае присутствия нейрона смещения.
- Оптимизатор — это метод достижения лучших результатов, помощь в ускорении обучения.
- Это алгоритм, используемый для незначительного изменения параметров, таких как веса и скорость обучения, чтобы модель работала правильно и быстро.
- **Импульсный оптимизатор**

Стохастический градиентный спуск - Stochasticy Gradient descent (SGD)

Берем случайные объекты, подаем их в модель и получаем предсказание, считаем функцию потерь, обновляем веса, повторяем снова до тех пор пока функция ошибки не окажется в точке минимума. Есть три варианта подачи объектов: по одному, весь набор, данных партиями (батчами, степенью числа 2 (32,64,128), это наиболее распространенная реализация).

Основная проблема - попадание функции в локальные минимумы вместо нахождения глобального. Для ее решения используются модификации стахастического градиента, использующие скользящее среднее градиентов:

- оптимизатор **Momentum**, учитывает прошлые градиенты для сглаживания обновления
- Оптимизатор **Nesterov Momentum**, его особенностью является вычисление градиента при обновлении
- **Адаптивные оптимизаторы**
- Цель адаптивных алгоритмов - отдельный learning rate для каждого из параметров. Чем чаще и сильнее меняется параметр, тем меньше его следующие изменения.
- **Среднеквадратичное распространение корня Adagrad**

Например, какой-то из нейронов в каждой итерации немного изменяет свои значения(0,0.1,0,0.1,0.2) и есть другой, который колеблется от 0 до 10, к примеру. Эти скачки нужно сгладить. Для этого мы по каждому нейрону храним его историю.

Экспоненциально затухающее среднее значение (RMSprop, root mean square)

Существенным свойством RMSprop является то, что вы не ограничены только суммой прошлых градиентов, но вы более ограничены градиентами последних временных шагов. RMSprop вносит свой вклад в экспоненциально

затухающее среднее значение прошлых «квадратичных градиентов». В RMSProp мы пытаемся уменьшить вертикальное движение, используя среднее значение.

Появляется новая зависимость размера шага от предыдущих. Уходит обязательность того, если 10 шагов назад был большой шаг, то сейчас должен быть меньший, и это позволяет выбраться из ямы. То есть он немного буксует в локальном минимуме, а потом снова начинает увеличиваться при необходимости, потому что зависимость от предыдущих шагов экспоненциальная.

- **Оптимизатор Adam**
- Adam — один из самых эффективных алгоритмов оптимизации в обучении нейронных сетей. Он сочетает в себе идеи RMSProp и оптимизатора импульса (momentum).
- Чем больше движемся в одну сторону, тем больше шаг и адаптация каждого параметра.
- Вместо того чтобы адаптировать скорость обучения параметров на основе среднего первого момента (среднего значения), как в RMSProp, Adam также использует среднее значение вторых моментов градиентов. В частности, алгоритм вычисляет экспоненциальное скользящее среднее градиента и квадратичный градиент

Функции активации

Определяют выходное значение нейрона в зависимости от результата взвешенной суммы входов и порогового значения.

Применяются к нейрону после того, просчитанному матричным умножением. Определяет, передавать ли полученное значение следующему

нейрону (активировать его) или нет, и модифицировать ли как-то полученное значение.

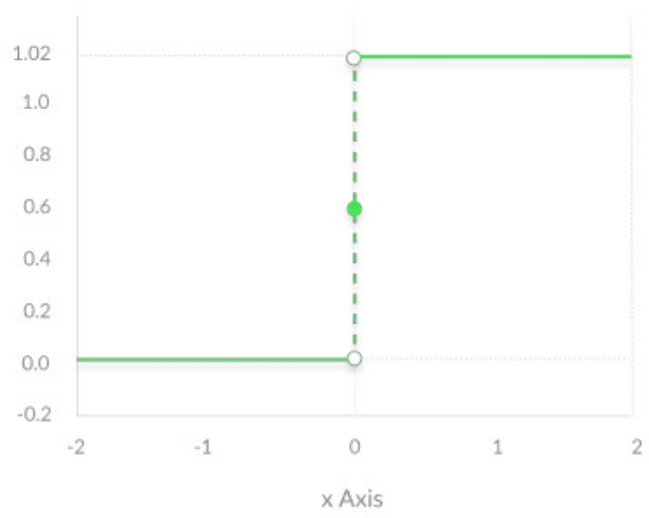
Рассмотрим нейрон, у которого взвешенная сумма равна z.

$$z = \sum_i w_i x_i + b,$$

Где wi и xi – вес и входное значение i-го входа, а b – смещение.

Ступенчатая

Ступенчатая функция (англ. binary step function) является пороговой функцией активации. То есть если z больше или меньше некоторого значения, то нейрон становится активированным. Такая функция отлично работает для бинарной классификации. Но она не работает, когда для классификации требуется большее число нейронов и количество возможных классов больше двух.



Линейная (linear)

Линейная функция (англ. linear function) представляет собой прямую линию, а это значит, что результат этой функции активации пропорционален переданному аргументу. В отличии от предыдущей функции, она позволяет

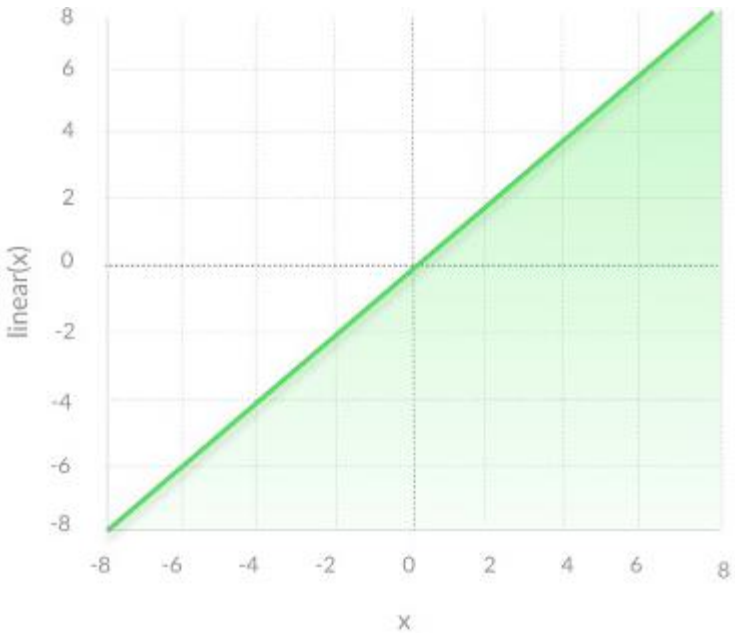
получить диапазон значений на выходе, а не только бинарные 0 и 1, что решает проблему классификации с большим количеством классов.

У линейной функции есть две основных проблемы:

1. Невозможность использования метода обратного распространения ошибки. Так как в основе этого метода обучения лежит градиентный спуск, а для того чтобы его найти, нужно взять производную, которая для данной функции активации — константа и не зависит от входных значений. То есть при обновлении весов нельзя сказать улучшается ли эмпирический риск на текущем шаге или нет.

2. Так как для каждого слоя выходное значение линейно, то они образуют линейную комбинацию, результатом которой является линейная функция. То есть финальная функция активации на последнем слое зависит только от входных значений на первом слое. А это значит, что любое количество слоев может быть заменено всего одним слоем, и, следовательно, нет смысла создавать многослойную сеть.

Главное отличие линейной функции от остальных в том, что ее область определения не ограничена: $(-\infty;+\infty)$. Следовательно, ее нужно использовать, когда выходное значение нейрона должно $\in R$, а не ограниченному интервалу.

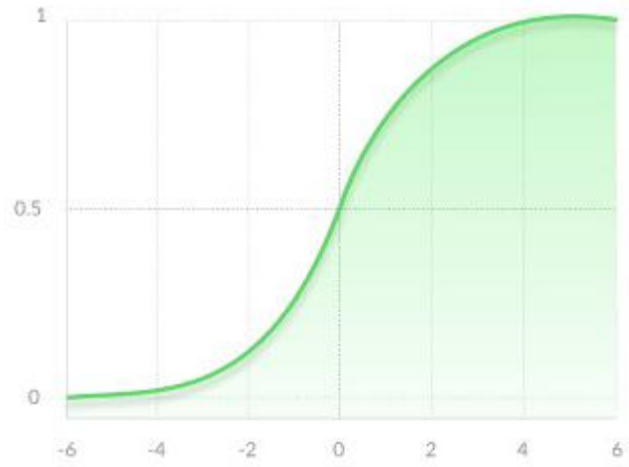


Сигмоидная функция (sigmoid)

Сигмоидная функция (англ. sigmoid function), которую также называет логистической (англ. logistic function), является гладкой монотонно возрастающей нелинейной функцией. Преобразует выходное значение нейрона в значение из диапазона [0, +1], смещает среднее значение с 0 и используется в случаях, когда нужно получить что-то похожее на вероятность.

Несмотря на множество сильных сторон сигмоидной функции, у нее есть значительный недостаток. Производная такой функции крайне мала во всех точках, кроме сравнительно небольшого промежутка - **проблема исчезающего градиента**. Это сильно усложняет процесс улучшения весов с помощью градиентного спуска. Более того, эта проблема усугубляется в случае, если модель содержит много слоев.

Что касается использования сигмоидной функции, то ее преимущество над другими — в нормализации выходного значения.

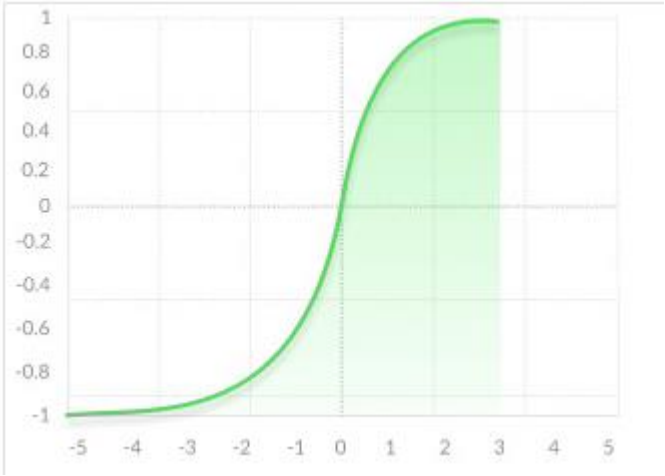


Функция гиперболического тангенса (tahn)

Функция гиперболического тангенса (англ. hyperbolic tangent). Преобразует выходное значение нейрона в значение из диапазона [-1, +1], не смещает среднее значение.

$$tanh(z) = \frac{2}{1+e^{-2z}} - 1 \ = 2 \cdot sigma(2z) - 1$$

Эта функция является скорректированной сигмоидной функцей, то есть она сохраняет те же преимущества и недостатки, но уже для диапазона значений (−1;1).



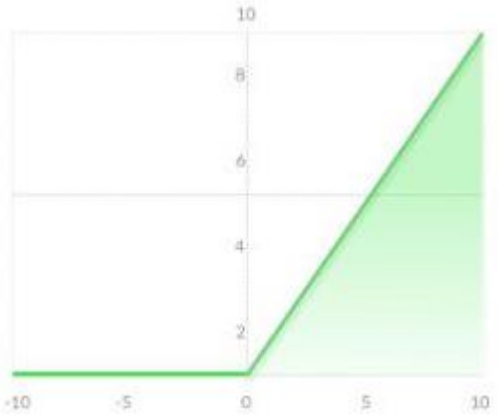
Функция ReLU

Функция ReLU (Rectified Linear Unit) — это наиболее часто используемая функция активации при глубоком обучении. Данная функция возвращает 0, если принимает отрицательный аргумент, в случае же положительного аргумента, функция возвращает само число. То есть она может быть записана как f(z)=max(0,z).

На первый взгляд может показаться, что она линейна и имеет те же проблемы что и линейная функция, но это не так и ее можно использовать в нейронных сетях с множеством слоев. Функция ReLU обладает несколькими преимущества перед сигмоидой и гиперболическим тангенсом:

- Очень быстро и просто считается производная. Для отрицательных значений — 0, для положительных — 1.
- Разреженность активации. В сетях с очень большим количеством нейронов использование сигмоидной функции или гиперболического тангенса в качестве активационный функции влечет активацию почти всех нейронов, что может сказаться на производительности обучения модели. Если же использовать ReLU, то количество включаемых нейронов станет меньше, в силу характеристик функции, и сама сеть станет легче.

У данной функции есть один недостаток, называющийся **проблемой умирающего ReLU**. Так как часть производной функции равна нулю, то и градиент для нее будет нулевым, а то это значит, что веса не будут изменяться во время спуска и нейронная сеть перестанет обучаться.



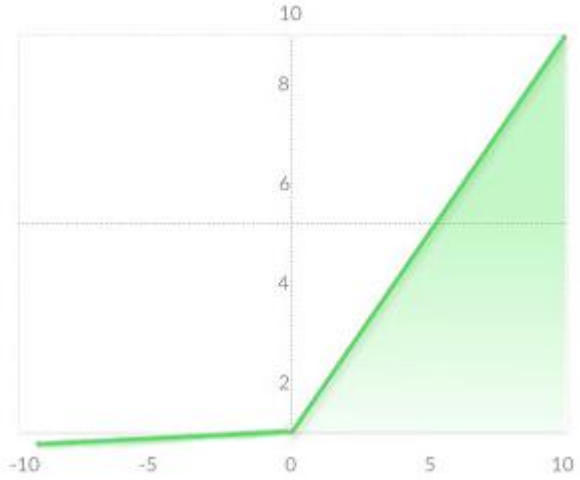
Функция Leaky ReLU

Одной из проблем стандартного ReLU является затухающий, а именно нулевой, градиент при отрицательных значениях. При использовании обычного ReLU некоторые нейроны умирают, а отследить умирание нейронов не просто. Чтобы решить эту проблему иногда используется подход ReLU с «утечкой» (leak) — график функции активации на отрицательных значениях образует не горизонтальную прямую, а наклонную, с маленьким угловым коэффициентом (порядка 0,01).

Такое небольшое отрицательное значение помогает добиться ненулевого градиента при отрицательных значениях. Однако, функция Leaky ReLU имеет некоторые недостатки:

- Сложнее считать производную, по сравнению со стандартным подходом (так как значения уже не равны нулю), что замедляет работу каждой эпохи.
- Угловой коэффициент прямой также является гиперпараметром, который надо настраивать.
- На практике, результат не всегда сильно улучшается относительно ReLU.

Стоит отметить, что помимо проблемы умирающих нейронов, у ReLU есть и другая — проблема затухающего градиента. При слишком большом количестве слоев градиент будет принимать очень маленькое значение, постепенно уменьшаясь до нуля. Из-за этого нейронная сеть работает нестабильно и неправильно. Leaky ReLU (LReLU) решает первую проблему, но в по-настоящему глубоких сетях проблема затухания градиента все еще встречается и при использовании этого подхода.



Функция Softmax

Softmax — это обобщение логистической функции (уравнение Ферхюльста) для многомерного случая.

Функция преобразует вектор x размерности K в вектор sigma той же размерности, где каждая координата sigma_i полученного вектора представлена вещественным числом в интервале [0,1] и сумма координат равна 1.

Координаты sigma_i вычисляются следующим образом:

$$\sigma(x)_i = \frac{e^{x_i}}{\sum_{k=1}^{k} e^{x_k}}$$

Функция Softmax применяется для задач классификации, когда количество возможных классов больше двух.

Loss-функция / функция потерь нейронной сети (функция ошибки) – математическая дифференцируемая функция, характеризующая разницу между «истинным» значением целевой переменной и предсказанным нейронной сетью значением.

Актуально для такой нейронной сети, которая обучается с использованием обучающего набора (датасета), который содержит примеры с истинными значениями: тегами, классами, показателями.

Средняя абсолютная ошибка (mean absolute error MAE)

Средняя абсолютная ошибка представляет из себя модуль разности двух значений. Если сеть обучается на наборе данных (а не на одном примере), для подсчета ошибки берется среднее значение по всем примерам.

Чтобы рассчитать MAE, надо взять разницу между предсказанными значениями и истинными, усреднить по всему набору данных.

$$MAE = \frac{1}{n} \cdot \sum_{i=1}^{n} |Yt - Yp|$$

где n = кол-во наблюдений, Yt - истинное значение, Yp - прогнозируемое значение

Среднеквадратичная ошибка (mean squared error MSE)

Средняя квадратичная ошибка (MSE): средняя квадратичная ошибка является наиболее распространенной функцией потерь. Функция потерь MSE

широко используется в линейной регрессии в качестве показателя эффективности (в паре с функцией активации linear).

Чтобы рассчитать MSE, надо взять разницу между предсказанными значениями и истинными, возвести ее в квадрат и усреднить по всему набору данных.

$$MSE = \frac{1}{n} \cdot \sum_{i=1}^{n} (Yt - Yp)^2$$

Кросс-энтропия (**categorical crossentropy CCE**)

Кросс-энтропия (или логарифмическая функция потерь – log loss). Кросс-энтропия измеряет расхождение между двумя вероятностными распределениями. Если кросс-энтропия велика, это означает, что разница между двумя распределениями велика, а если кросс-энтропия мала, то распределения похожи друг на друга.

$$\text{Loss} = -\sum_{i=1}^{\substack{\text{output} \\ \text{size}}} y_i \cdot \log \hat{y}_i$$

где:

- output size - количество наблюдений
- Yi - истинное значение
- ^Yi - прогнозируемое значение

Бинарная классификация (binary crossentropy BCE)

Это функция ошибок, которую можно использовать для количественной оценки разницы между двумя распределениями вероятностей. Она говорит о том, что если у нас есть события и вероятности, насколько вероятно, что события произойдут на основе вероятностей? Если это очень вероятно, у нас малая кросс-энтропия, а если маловероятно, у нас высокая кросс-энтропия.

Формула имеет вид:

$$\text{Loss} = -\frac{1}{\text{output size}} \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i + (1 - y_i) \cdot \log (1 - \hat{y}_i)$$

где:

- output size - количество наблюдений
- Yi - истинное значение
- ^Yi - прогнозируемое значение

Вывод: в ходе лабораторной работы были освоены навыки по основам создания нейронных сетей и их более глубокого изучения.